

1 Interpreted Java Service

1.1 Introduction

Interpreted Java services are web services that are converted by an interpreter to SELECT SQL statements. The result is converted to a web service result, once the SQL statement has been executed.

The definition for the interpreter is created in XML files using the repository client.

If you create new services, use the type `InterpretedJavaService2` instead of the `InterpretedJavaService`.

The `InterpretedJavaService2` type is prepared for the future streaming of data and offers more options for Java user exits.



As long as no Java user exits are used, it is still easy to convert the `InterpretedJavaService` type into the `InterpretedJavaService2` type by simply changing the service type. There are the following differences for services without Java user exits.

- The column `DataObjectName` for `InterpretedJavaService2` is not required in the repository data of service and should be set to empty.
- The column `parameterReference` is not required anymore for the repository data of the `DataObjects` and should be set to empty.

1.2 Definition

An interpreted Java service is defined in the repository (for further information on the required values and their meaning, refer section "repository data" below).

The service domain can be exported as XML file, once the definition has been completed. The resulting files (the ones relevant to the interpreter) are **<Domain>.Configuration.xml** and **<Domain>.do.xml**.

1.3 Storage in a server

Both files are located on a server under `jdir\MOC\<SYSTEM>\listInterpreter\<Scope>` or `jhydradir\MOC\<SYSTEM>\listInterpreter\<Scope>`

The scope has one of the following values: standard, custom or local.

1.4 Available Special Parameters

Each interpreted Java service includes special parameters that are always available and can always be used. Subject to how the interpreted Java service is customized in the file <Domain>.do.xml, the parameters affect processing or are ignored.

These parameters are available:

Name	Data type	Operators	Description
checkresponsibilityarea	boolean	EQUAL	Is only effective if checkRespAreaMode and checkRespAreaField are configured. If false, the responsibility area will not be checked. If true, it is checked. The default value is true.
checkresponsibilityareafunctions	String[]	EQUAL, IN	Is only effective, if checkRespAreaMode and checkRespAreaField are configured and checkresponsibilityarea == true. This parameter controls which functions are checked by the responsibility area. The default value is select. The following functions can be indicated: create (vab_tab.anlegen='J') delete (vab_tab.loeschen='J') select (vab_tab.anzeigen='J') update (vab_tab.aendern='J') use (vab_tab.verwenden='J')
longtermdata	boolean	EQUAL	Is only effective if tableClauseLongterm is configured. If false, no long-term data is used. If true, it is checked. The default value is false.

1.5 Repository data

1.5.1 Tab Services

Name	Meaning	Optional
Domain	The domain of the service (e.g. BOPerson)	
Name	The complete service name, i.e. Domain.Function (e.g. BOPerson.list)	
Service Function	The function of the service (e.g. list)	
Service Type	The type - for interpreted Java services: fixed InterpretedJavaService	
Description (German)	Brief (internal) description of the service	

1.5.2 Tab ServiceParameter

Name	Meaning	Optional
Domain	The domain of the service (e.g. BOPerson)	
Service Function	The function of the service (e.g. list)	
Service	The complete service name, i.e. Domain.Function (e.g. BOPerson.list)	
Acronym	Acronyms (e.g. person.id) have to be unique for service and result set.	
Web service type	The data type of the parameter (decimal, integer, string, boolean, datetime)	
DB table	The table that is used to select the value for the acronym	

DB field	The field from which the value for the acronym is to be selected This is either just the field name or the expression (if it is a calculated field) including placeholder for the table alias (e.g. <code>hydadm.get_datetime (%1\$s.bearb_date, %1\$s.bearb_time)</code> or <code>{fn substring (%1\$s.field, 2, 1)}</code>). The placeholder for the table alias is always <code>"%1\$s"</code> .	
DB Alias	The table alias for the table that is used to select the value for the acronym	
Conversion Method	Here you can specify transformations for input and result parameters (e.g. conversion bool to J/N and vice versa or the correct filtering for datetime fields that consist of two fields in the database). Possible transformations are described elsewhere.	X
Can ...	Have to be set for filter parameters (for Boolean only Can Equal; for string all and for the others, everything except Can Like and Can Like or null)	X (if only Result)
IsFilterParameter	Specifies whether or not the field is a filter field. A filter parameter including its operator is directly converted into an SQL fragment	X (if only Result)
IsResult	Specifies whether or not it is a Result	X (if only Filter)
IsSpecialParameter	Specifies whether or not a field is a special parameter. At the moment, standard processing only supports the above-mentioned special parameters. Further parameters can be used in exits. Special parameters are "options" that cannot directly be converted into an SQL fragment of the type <code><DB field> <Operator> <Value></code> .	X
IsMandatory	Specifies whether or not it is a mandatory field. If this is true and the parameter is missing, an error message is generated at runtime. Is currently only checked for special parameters.	X

InputAsArray	Specifies whether the field also supports arrays as input parameters. (e.g. the operators IN or BETWEEN require an array as input parameter)	
DataObjectName	Name of the interpreted Java Service. Used as reference for the ...do.xml configuration	
ConditionalFieldKey	This field specifies if a DB field is only conditionally available. The condition as to whether the field is available is checked using the Configuration Manager (see Configuration Manager in the section "server"). The feature key of the Configuration Manager (feature set) has to be entered in this field within the repository for checking.	X
DBFieldAlternative	Only relevant if it is a conditional field. Here you can enter the alternative value. This value can be a figure, null, 'string', {fn ...}, or even another field / subselect. "%1\$s." NEEDS to be entered for the alias if it is another field or subselect! The default value is null if nothing is entered.	X

1.5.3 Tab Dataobjects

name

Name of the data source. References to the field DataObjectName for the repository (to connect the ServiceParameter)

parameterReference

References to the service name in order for the correct parameters to be determined

orderBy (optional)

Specifies sorting (with the real alias and not %1\$s. in front of the field name)

Please note: Do not use if "groupByCols" is applied. This might lead to SQL errors if sorting is based on a field that is not included in the "group by" clause (in case the client has not requested it).

groupByCols (optional)

Specifies the "group by" fields (in the order) for the SQL statement.

The value includes a list of acronyms including their database field (with the real alias and not %1\$s. in front of the field name). The interpreter only adds the fields requested by the client (including their acronyms) to the group by clause.

For example:

Acronym1=alias.field1|Acronym2=alias.field2|...

filterBy (optional)

Specifies fixed filters (with the real alias and not %1\$s. in front of the field name)

checkRespAreaMode (optional)

Checks the responsibility area of the current user.

Modes:

- „none“: no check
- „direct“: Check performed via a field of the data source (joined to vab_tab). Use of -- DEFAULT-- if empty or zero.
- „directnotempty“: Check performed via a field of the data source (joined to vab_tab)
- „person“: Check performed via the responsibility area (VBA) of a person.
- „machine“: Check performed via the VAB assigned to the machine.

checkRespAreaField (optional)

Specifies the field of the main table which contains the VAB for direct/directnotempty.

Join field if person or machine

(with the real alias and not %1\$s. in front of the field name)

checkRespAreaDefaultValue (optional)

Specifies the default value for checking the responsibility area (if the parameter has not been specified by the client). The default value is true. Valid values are true and false.

dataTabLabel (optional)

Name of the result set. The field normally remains empty and is only required if special processing is performed by further user exits.

tableClause (optional)

Specifies the table clause without the key word FROM (only relevant if several tables are used). If nothing is entered here, the first table and its alias found for a service parameter will be used as the tableClause.

tableClauseLongterm (optional)

Specifies the table clause for long-term data (only relevant if several tables are used or if long-term data exist). If nothing is entered the tableClause is used as tableClauseLongterm. If this one is neither indicated, the first table and its alias found for a service parameter will be used.

conditionalLongtermKey (optional)

This value specifies if the long-term data tables are only conditionally available. The condition as to whether the tables are available is checked by the Configuration Manager. The feature key of the Configuration Manager has to be entered here for checking.

mergeOnAttributeLevel (optional)

Attribute available as of SP11:

This attribute controls how individual DataObjects are merged over several scopes (a single DataObject is identified by the attribute "name").

- If the attribute mergeOnAttributeLevel does not exist or the value is not equal "Y", the behavior is the same than before SP11 (backward compatibility). This means that the whole configuration of the DataObjects (not the whole file, but only the entire row) is completely overwritten by a higher scope. For example, if a filterBy is introduced in the custom scope, the complete DataObject in the standard scope is replaced. If the standard is then extended, the standard extension is not applied in the custom scope.
- If the attribute is set to "Y", the merge behavior changes and one attribute is merged after the other and not the complete DataObject at a time. Refer to the following subchapter for details. (This setting is the default setting for new DataObject configurations as of SP11; existing older configurations are not changed).

You can find details and examples in the next subchapter.

1.5.3.1 Rules for the merge of SQL attributes on attribute level

Only "name" and "parameterReference" are mandatory attributes if you merge DataObjects on attribute level. The other values are taken over from the lower scope.

The following applies for most attributes:

- If the attribute is empty, the value of the lower scope **is taken over**.
- If the attribute is populated, the value of the lower scope **is overwritten**.

These SQL attributes are an exception:

- tableClauseLongterm
- tableClause
- filterBy
- groupByCols
- orderBy

The following applies with these SQL attributes:

- If the attribute is empty, the value of the lower scope **is taken over**.
- If the attribute is populated, the value of the lower scope **is written in front** (the value of the higher scope is written behind the value of the lower scope). A specific separator is used for the separation:
 - o tableClauseLongterm: a space character
 - o tableClause: a space character
 - o filterBy: an "AND"; in addition the lower scope is put in parentheses. If the higher scope does not include a key word, also the higher scope is put in parentheses.
 - o orderBy: a comma (",")
 - o groupByCols: a pipe ("|")
- If the attribute includes the key word \$NO_LOWER_SCOPE\$, the value of the lower scope is completely replaced by the value of the attribute.

If the attribute includes the key word \$LOWER_SCOPE_VALUE\$, the lower scope is copied to exactly this position (with the specific separator).

1.5.3.2 Examples of the merge of SQL attributes on attribute level

As of SP11

The following examples only refer to the attributes tableClauseLongterm, tableClause, filterBy, orderBy and groupByCols. The other attributes follow a very simple pattern: if the attribute in a higher scope is populated, the lower scope is replaced. If the attribute in the higher scope is not populated, the lower scope is used.

In the following, you can find examples of the behavior, if you merge on attribute level (mergeOnAttributeLevel=Y). We used "filterBy" in our examples. The first row is the lower scope (e.g. standard), the second row is the higher scope (e.g. custom) and the third row is the result of the merge.

```
"xy.field_x = '42'",
"foo.field_y = '24'",
"(xy.field_x = '42') and (foo.field_y = '24')"
```

```
"xy.field_x = '42'",
"",
"xy.field_x = '42'"

"",
"foo.field_y = '24'",
"foo.field_y = '24'"

"xy.field_x = '42'",
"foo.field_y$LANG = 'foo' $LOWER_SCOPE_VALUE$",
"foo.field_y$LANG = 'foo' and (xy.field_x = '42')"
```

```
"xy.field_x = '42'",
"foo.field_y = '24' $LOWER_SCOPE_VALUE$",
"foo.field_y = '24' and (xy.field_x = '42')"
```

```
"",
"foo.field_y = '24' $LOWER_SCOPE_VALUE$"
```

```

" foo.field_y = '24'"

"xy.field_x = '42'",
"foo.field_y = '24' and $LOWER_SCOPE_VALUE$ bar.field_z = 'world'",
"foo.field_y = '24' and (xy.field_x = '42') and bar.field_z = 'world'"

"",
"foo.field_y = '24' and $LOWER_SCOPE_VALUE$ bar.field_z = 'world'",
"foo.field_y = '24' and bar.field_z = 'world'"

"xy.field_x = '42'",
"$LOWER_SCOPE_VALUE$ foo.field_y = '24'",
"(xy.field_x = '42') and foo.field_y = '24'"

"",
"$LOWER_SCOPE_VALUE$ foo.field_y = '24'",
" foo.field_y = '24'"

"xy.field_x = '42'",
"foo.field_y = '24' $NO_LOWER_SCOPE$",
" foo.field_y = '24'"

"",
"foo.field_y = '24' $NO_LOWER_SCOPE$",
" foo.field_y = '24'"

"xy.field_x = '42'",
"foo.field_y = '24' and $NO_LOWER_SCOPE$ bar.field_z = 'world'",
"foo.field_y = '24' and bar.field_z = 'world'"

"",
"foo.field_y = '24' and $NO_LOWER_SCOPE$ bar.field_z = 'world'",
"foo.field_y = '24' and bar.field_z = 'world'"

"xy.field_x = '42'",
"$NO_LOWER_SCOPE$ foo.field_y = '24'",
" foo.field_y = '24'"

"",
"$NO_LOWER_SCOPE$ foo.field_y = '24'",
" foo.field_y = '24'"

"satz_art = 'U'",
"$NO_LOWER_SCOPE$ masch_nr = '4711' $LOWER_SCOPE_VALUE$",
" masch_nr = '4711' and (satz_art = 'U')"

"satz_art = 'U'",
"masch_nr = '4711' $NO_LOWER_SCOPE$LOWER_SCOPE_VALUE$",
"masch_nr = '4711' LOWER_SCOPE_VALUE$" => invalid SQL

"satz_art = 'U'",
"masch_nr = '4711' $LOWER_SCOPE_VALUE$NO_LOWER_SCOPE$",
"masch_nr = '4711' (satz_art = 'U') andNO_LOWER_SCOPE$" => invalid SQL

"satz_art = 'U'",
"masch_nr = '4711' $LOWER_SCOPE_VALUE$ $LOWER_SCOPE_VALUE$",
"masch_nr = '4711' (satz_art = 'U') and $LOWER_SCOPE_VALUE$" => invalid SQL

"satz_art = 'U'",
"$LOWER_SCOPE_VALUE$ masch_nr = '4711' $LOWER_SCOPE_VALUE$",
"(satz_art = 'U') and masch_nr = '4711' $LOWER_SCOPE_VALUE$" => invalid SQL

```

1.6 Exits

Exits provide the entry points to enable changes to the defined behavior by programming.



Instead of the user exits and program exits presented below, use the [GlobalExits](#). The GlobalExits ensure the greatest possible compatibility in the further development of the system, as they are supported equally for all service types.

1.6.1 Available user exits

User exits provide entry points that are not modified by releases. In case of releases, the interfaces are respected and preserved by MPDV in a backwards compatible manner.

1.6.1.1 sdiModifyColumnConfigurator

With this user exit, the interpreted Java service provides the application developer with an option to modify/extend the column configurator before it is executed.

1.6.1.2 sdiModifyResultList

With this user exit, the interpreted Java service provides the application developer with an option to modify/extend the service result after it is executed.

1.6.2 Available program exits

Program exits offer extended functionality that is not backwards compatible. Changes can be made with every update. Users of program exits must test if their modifications still work after an update.

Therefore, program exits are of limited value with modifications.

1.6.2.1 sdiModifyColumnMap

With this program exit, the interpreted Java service provides the application developer with an option to modify/extend standard assignment of acronym to DB table before it is executed.

1.6.2.2 sdiAugmentSql

With this program exit, the interpreted Java service provides the application developer with an option to modify the generated SQL shortly before it is executed.

1.6.2.3 sdiModifySql

As of SP8: with this program exit, the SQL from the generator or of lower scopes can be overwritten explicitly.

1.6.3 Specifications for the implementation class

Package name: You must include the class in a package that consists of the domain name (in lower case letters). Further subpackages are **not** allowed.

Example: The service is requested "MDUserAccountRules.list", consequently the package is called "mduseraccountrules"

Class name: The class must have a name of the following structure: domain name in lower case letters, whereas the first letter is written in capital letters, the name of the service function follows and is written in lower case letters, whereas the first letter is once again written in upper case.

Example: The service is requested "MDUserAccountRules.list", consequently the class is called "MduseraccountrulesList"



The following definition applies for customized class names:

Customized names include "_" (see naming conventions)

After "_" the first letter is changed to upper case and the underscore is skipped

Example: U_CUST_Units_sample.list => UCustUnitsSampleList

Implemented interfaces / methods: no specifications

Other: The class must have a default constructor without parameters

Compilation: The Jar files *MpdvDomCoreSdiCompileLib.jar* and *MpdvDomCoreUserExitCompileLib.jar* must be included in the class path for the compile process.

Deployment: The class file of the exit must be stored in the directory <JDIR>/MOC/<System>/userexit/<scope> including package directory structure.

Example: Exit sdiAugmentSql for service "MDUserAccountRules.list":

```
package mduseraccountrules;
```

```
import de.mpdv.customization.userExit.IUserExitParam;
```

```
/**
 * Sample user exit
```

```

*
*
*/
public class MduseraccountrulesList
{
    public void sdiAugmentSql(final IUserExitParam param)
    {
        // TODO implementation
    }
}

```

Directory structure on the server (System 1, scope custom):

<JDIR>/MOC/1/userexit/custom/mduseraccountrules/MduseraccountrulesList.class

1.6.4 Interfaces

1.6.4.1 Class: InterpretedJavaServiceUeContext

de.mpdv.sdi.data.ue.InterpretedJavaServiceUeContext
-hydraNow: Calendar
-userId: String
+getHydraNow(): Calendar
+getUserId(): String

This context class provides data that is generally useful for interpreted Java services as regards to exits.

Field	Description
hydraNow	The property "hydraNow" is a time stamp created at the beginning of web service processing and, as a result, can be used as reference time stamp for the current web service call.
userId	Includes the user logged on to the client.

1.6.4.2 Program exit: sdiModifyColumnConfigurator

Parameter key in IUserExitParam:

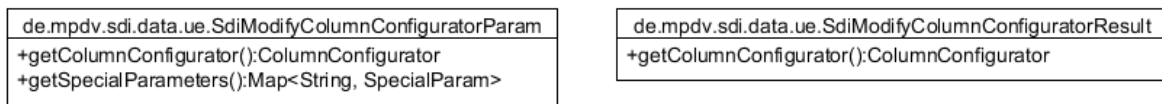
Key name	Type	Description
param	SdiModifyColumnConfiguratorParam	Parameter structure for the user exit

context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiModifyColumnConfiguratorResult	Result structure for the user exit

Class diagram of parameter and result structures:



1.6.4.3 Class SdiModifyColumnConfiguratorParam

Field	Type	Description
columnConfigurator	ColumnConfigurator	Includes the columns requested by the client
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"

1.6.4.4 Class SdiModifyColumnConfiguratorResult

Field	Type	Description
columnConfigurator	ColumnConfigurator	Includes the requested columns after processing in user exit

1.6.4.5 Program exit: sdiModifyColumnMap

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiModifyColumnMapParam	Parameter structure for the program exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiModifyColumnMapResult	Result structure for the program exit

Class diagram of parameter and result structures:

de.mpdv.sdi.data.ue.SdiModifyColumnMapParam
+getColumnMap():Map<String, String>
+getSpecialParameters():Map<String, SpecialParam>

de.mpdv.sdi.data.ue.SdiModifyColumnMapResult
+getColumnMap():Map<String, String>

1.6.4.6 Class SdiModifyColumnMapParam

Field	Type	Description
columnMap	Map<String, String>	Includes the assignment of acronym => table column (including alias)
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"

1.6.4.7 Class SdiModifyColumnMapResult

Field	Type	Description
-------	------	-------------

columnMap	Map<String, String>	Includes the assignment of acronym => table column (including alias)
-----------	---------------------	---

1.6.4.8 Program exit: sdiAugmentSql

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiAugmentSqlParam	Parameter structure for the program exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiAugmentSqlResult	Result structure for the program exit

Class diagram of parameter and result structures:

de.mpdv.sdi.data.ue.SdiAugmentSqlParam
+getSelect():String +getFrom():String +getWhere():String +getGroupBy():String +getOrderBy():String +getSpecialParameters():Map<String, SpecialParam>

de.mpdv.sdi.data.ue.SdiAugmentSqlResult
+getFromSuffix():String +getWhereSuffix():String +getGroupBySuffix():String +getOrderBySuffix():String

1.6.4.9 Class SdiAugmentSqlParam

Field	Type	Description
-------	------	-------------

select	String	SELECT clause created based on the configuration (only includes the column part up to FROM, only without the key word SELECT)
from	String	FROM clause created based on the configuration (without the key word FROM)
WHERE	String	WHERE clause created based on the configuration (without the key word WHERE)
groupBy	String	GROUP BY clause created based on the configuration
orderBy	String	ORDER BY clause created based on the configuration
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"

1.6.4.10 Class SdiAugmentSqlResult

Field	Type	Description
fromSuffix	String	Suffix for the FROM clause created in the program exit or NULL
whereSuffix	String	Suffix for the WHERE clause created in the program exit or NULL
groupBySuffix	String	Suffix for the GROUP BY clause created in the program exit or NULL
orderBySuffix	String	Suffix for the ORDER BY clause created in the program exit or NULL

1.6.4.11 User exit: sdiModifyResultList

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiModifyResultListParam	Parameter structure for the user exit
context	InterpretedJavaServiceUeContext	Context structure for all user exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in user exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiModifyResultListResult	Result structure for the user exit

Class diagram of parameter and result structures:

de.mpdv.sdi.data.ue.SdiModifyResultListParam
+getColumnConfigurator():ColumnConfigurator
+getDataTables():List<IDataTable>
+getSpecialParameters():Map<String, SpecialParam>

de.mpdv.sdi.data.ue.SdiModifyResultListResult
+getDataTables():List<IDataTable>

1.6.4.12 Class SdiModifyResultListParam

Field	Type	Description
columnConfigurator	ColumnConfigurator	Includes the columns requested by the client
dataTables	List<IDataTable>	Includes the data table(s) generated as service result by the interpreter
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"

1.6.4.13 Class SdiModifyResultListResult

Field	Type	Description
dataTables	List<IDataTable>	Includes the data table(s) after processing in the user exit that the service supplies as result to the client

1.6.4.14 Program exit: sdiModifySql

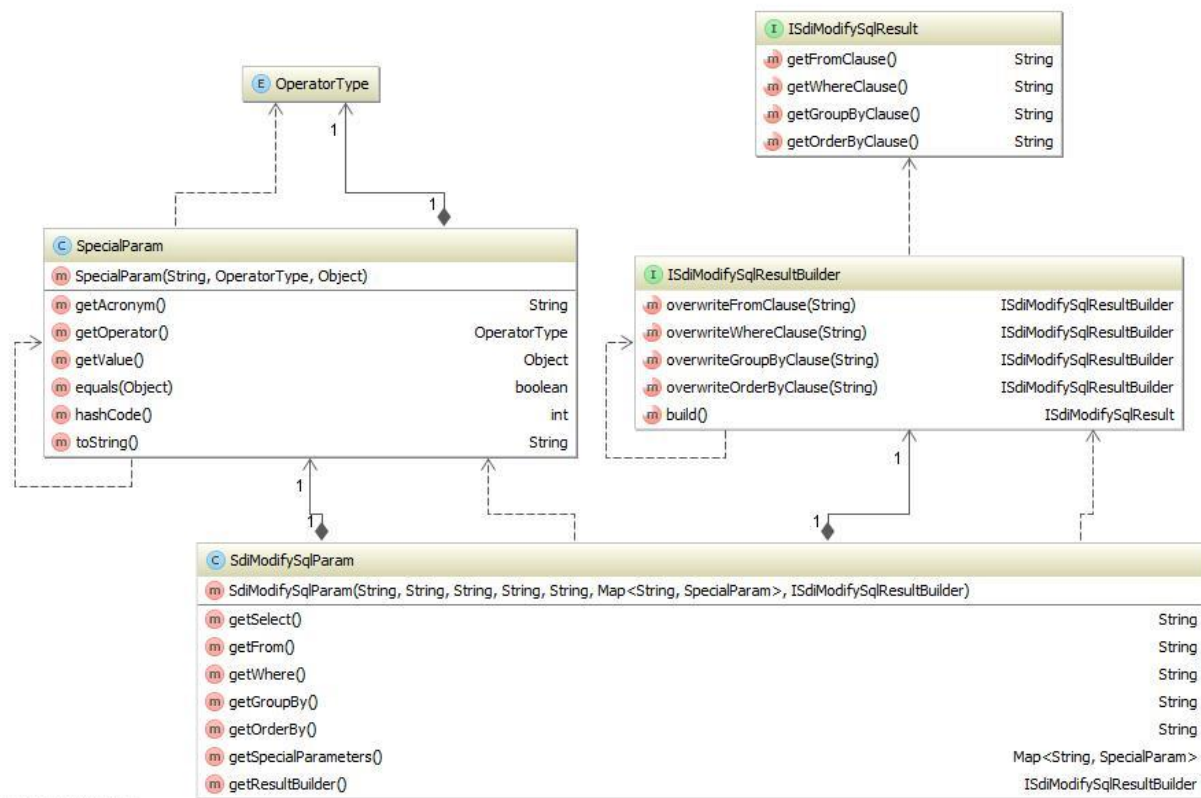
Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiModifySqlParam	Parameter structure for the program exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Return key in IUserExitParam:

Key name	Type	Description
result	ISdiModifySqlResult	Result structure of the program exit

Class diagram of parameter and result structures:



Powered by yFiles

1.6.4.15 Class SdiModifySqlParameter

Field	Type	Description
select	String	SELECT clause created based on the configuration (only includes the column part up to FROM, only without the key word SELECT)
from	String	FROM clause created based on the configuration (without the key word FROM)
WHERE	String	WHERE clause created based on the configuration (without the key word WHERE)
groupBy	String	GROUP BY clause created based on the configuration

orderBy	String	ORDER BY clause created based on the configuration
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"

1.6.4.16 Interface ISdiModifySqlResultBuilder

Method	Description
overwriteFromClause():ISdiModifySqlResultBuilder	Overwrites FROM clause. Only call this method if you really want to overwrite the FROM clause!
overwriteWhereClause():ISdiModifySqlResultBuilder	Overwrites WHERE clause. Only call this method if you really want to overwrite the WHERE clause!
overwriteGroupByClause():ISdiModifySqlResultBuilder	Overwrites GROUP BY clause. Only call this method if you really want to overwrite the GROUP BY clause!
overwriteOrderByClause():ISdiModifySqlResultBuilder	Overwrites ORDER BY clause. Only call this method if you really want to overwrite the Order BY clause!
build():ISdiModifySqlResult	Creates the result structure of the program exit.

1.6.4.17 Interface ISdiModifySqlResult

Method	Description
--------	-------------

getFromClause(): String	FROM clause of the program exit if the clause is overwritten there. Otherwise it is the original clause.
getWhereClause(): String	WHERE clause of the program exit if the clause is overwritten there. Otherwise it is the original clause.
getGroupByClause(): String	GROUP BY clause from the program exit if it was overwritten there, otherwise the original clause.
getOrderByClause(): String	ORDER BY clause of the program exit if the clause is overwritten there. Otherwise it is the original clause.